ShopHub / Legacy Ecommerce Knowledge Bytes

A progressive codebase learning guide

36 focused lessons covering architecture, backend services, frontend state, authentication, cart and checkout workflows, testing, and migration decisions.

Learning principle

Understand the whole system first, then learn how individual files and responsibilities connect. This guide separates the legacy MVC application from the modern Vue and ASP.NET Core implementation.

Prepared from the supplied repository and its documentation. Runtime behavior and test status should be verified in the target environment.

How to use this guide

Read the bytes in order. This ZIP contains **two related but distinct implementations**: the original ASP.NET MVC 5 / EF6 application, and a newer ShopHub implementation with a Vue SPA and ASP.NET Core API. The newer implementation is under `backend/` and `frontend/`; the older four-project application is at the repository root (`Ecommerce.Web`, `Ecommerce.Services`, `Ecommerce.Data`, `Ecommerce.Core`). Do not confuse their entry points, database models, authentication, or cart implementations.

Evidence convention: “Observed” means supported directly by files inspected. “Inference” means a likely purpose or relationship, not a guarantee. This guide explains the principal source files and workflows rather than generated files, binaries, dependency caches, and every individual asset. No secrets from environment files are reproduced.

Byte 1: What this repository is

Builds on: None — starting point

Connected to: `README.md`, `docs/legacy-architecture.md`, `docs/architecture.md`, `docs/migration-notes.md`

File / Location: Repository root

In plain terms: This is a ShopHub e-commerce codebase that preserves an older MVC application and also contains a newer API-driven implementation. Both serve a storefront concept, but they use different application architectures.

Why it exists: The legacy application is the reference implementation/domain source; the modern implementation reimplements the shopping experience with an API and SPA.

How it fits into the project: Start with `README.md` and the docs before opening code. `docs/migration-notes.md` describes which behaviors were carried over or deliberately changed.

The code:

```
Legacy MVC app: Ecommerce.Web → Ecommerce.Services → Ecommerce.Data → SQL Server
Modern app:     frontend (Vue) → backend/Ecommerce.Api → backend/Ecommerce.Infrastructure → separate SQL
  Server DB
```

What is happening: The two paths should be treated as separate runnable applications. The modern documentation says the modern database is separate from `LegacyEcommerceDb`; do not point both implementations at the same database without a deliberate migration plan.

Byte 2: Technology stack of the modern implementation

Builds on: Byte 1

Connected to: `backend/Ecommerce.sln`, `backend/Ecommerce.Api/Ecommerce.Api.csproj`, `backend/Ecommerce.Infrastructure/Ecommerce.Infrastructure.csproj`, `frontend/package.json`

File / Location: `backend/`, `frontend/`

In plain terms: The modern backend uses C# and ASP.NET Core Minimal APIs on .NET 10, Entity Framework Core, and SQL Server; the frontend uses Vue 3, TypeScript, Vite, Pinia, Vue Router, and Axios.

Why it exists: The browser UI is separated from backend business operations and communicates with the API over HTTP/JSON.

How it fits into the project: The API exposes endpoints; Core defines domain contracts; Infrastructure implements persistence and services; Vue views use API modules and stores.

The code:

```
<!-- backend/Ecommerce.Api/Ecommerce.Api.csproj -->
<TargetFramework>net10.0</TargetFramework>
```

What is happening: The package manifests and project files are the evidence for the stack. The old MVC app instead uses .NET Framework 4.7, MVC 5, Razor, jQuery/Bootstrap, EF6 Database First, and SQL Server.

Byte 3: Repository layout and the duplicate names

Builds on: Bytes 1–2

Connected to: `Ecommerce.Core/`, `Ecommerce.Data/`, `Ecommerce.Services/`, `Ecommerce.Web/`, `backend/`, `frontend/`, `db/`, `docs/`

File / Location: Repository root

In plain terms: Several folders share names such as `Ecommerce.Core`, but they are not interchangeable. The root-level folders belong to the legacy MVC solution; `backend/Ecommerce.Core` belongs to the modern API solution.

Why it exists: The repository retains legacy code while building a separate replacement implementation.

How it fits into the project: When navigating or running commands, first decide which solution you are working on: root `LegacyEcommerce.sln` or `backend/Ecommerce.sln`.

The code:

```
Ecommerce.Core/                # legacy domain and view models
backend/Ecommerce.Core/        # modern entities, DTOs, interfaces, domain helpers
Ecommerce.Data/                # legacy EF6 EDMX/repositories
backend/Ecommerce.Infrastructure/ # modern EF Core/persistence/services
```

What is happening: This duplicated naming is a major source of confusion. Keep file paths explicit in notes, IDE searches, and terminal commands.

Byte 4: The legacy architecture

Builds on: Byte 3

Connected to: `docs/legacy-architecture.md`, `LegacyEcommerce.sln`, `Ecommerce.Web/Global.asax.cs`

File / Location: `Ecommerce.Web/`, `Ecommerce.Services/`, `Ecommerce.Data/`, `Ecommerce.Core/`

In plain terms: The legacy application is a server-rendered monolith split into four projects. MVC controllers and Razor views live in Web; use-case logic lives in Services; EF6 and repositories live in Data; shared entities, view models, and interfaces live in Core.

Why it exists: The layers separate UI, business rules, persistence, and shared contracts.

How it fits into the project: A request typically enters an MVC controller, calls a service, then a repository/EF context, and returns a Razor view or JSON response.

The code:

```
Browser → Ecommerce.Web → Ecommerce.Services → Ecommerce.Data → SQL Server
                    ■ Ecommerce.Core contracts/entities ■
```

What is happening: The legacy design uses Unity for dependency injection and OWIN/ASP.NET Identity 2 cookie authentication. This is different from the modern API's JWT-based design.

Byte 5: The modern architecture

Builds on: Bytes 2–4

Connected to: `docs/architecture.md`, `backend/Ecommerce.Api/Program.cs`, `backend/Ecommerce.Infrastructure/ServiceCollectionExtensions.cs`

File / Location: `backend/`

In plain terms: The modern frontend is a single-page application. It sends API requests to ASP.NET Core, whose endpoint handlers call service interfaces implemented in Infrastructure, which uses EF Core to access the modern SQL Server database.

Why it exists: The API becomes the contract between the independent UI and backend.

How it fits into the project: Core contains entities/DTOs/interfaces/domain helpers; API contains HTTP concerns; Infrastructure contains persistence and service implementations.

The code:

```
Vue SPA → JSON HTTP → Minimal API endpoint → service → EcommerceDbContext → ShopHubModernDb
```

What is happening: The docs describe middleware including exception handling, logging, CORS, rate limiting, concurrency controls, authentication, and authorization. Exact middleware order should be confirmed in `Program.cs` when debugging a request.

Byte 6: Modern backend startup

Builds on: Byte 5

Connected to: `backend/Ecommerce.Api/Program.cs`, `Ecommerce.Infrastructure/ServiceCollectionExtensions.cs`

File / Location: `backend/Ecommerce.Api/Program.cs`

In plain terms: `Program.cs` is the modern API composition root: it creates the web app, configures logging/security/middleware, registers infrastructure, and maps endpoints.

Why it exists: ASP.NET Core needs one place to assemble the application's services and HTTP pipeline.

How it fits into the project: It calls `AddEcommerceInfrastructure`, configures JWT authentication and CORS, and later maps endpoint groups and health/OpenAPI behavior.

The code:

```
builder.Services.AddEcommerceInfrastructure(builder.Configuration);
builder.Services.AddJwtAuthentication(builder.Configuration);
```

What is happening: When startup fails, inspect configuration and registrations here before debugging individual endpoints. `Program.cs` is not where most business rules should live.

Byte 7: Dependency injection and service registration

Builds on: Byte 6

Connected to: `backend/Ecommerce.Infrastructure/ServiceCollectionExtensions.cs`, `backend/Ecommerce.Core/Interfaces/Services/`

File / Location: `backend/Ecommerce.Infrastructure/ServiceCollectionExtensions.cs`

In plain terms: Dependency injection creates the service objects that endpoints depend on. The registration extension connects interfaces such as `ICartService` to concrete classes such as `CartService`.

Why it exists: Endpoint handlers can depend on abstractions rather than manually constructing database and service objects.

How it fits into the project: It configures `EcommerceDbContext`, password hashing, token services, catalog/cart/order/account/admin/auth services, and database seeding.

The code:

```
services.AddScoped<ICartService, CartService>();
services.AddScoped<IOrderService, OrderService>();
services.AddScoped<IAuthService, AuthService>();
```

What is happening: Scoped services generally live for one request scope. If a handler cannot resolve a service, check its interface, implementation, and registration.

Byte 8: Core entities, DTOs, and interfaces

Builds on: Bytes 5–7

Connected to: `backend/Ecommerce.Core/Entities/`, `Dtos/`, `Interfaces/Services/`, `Domain/`

File / Location: `backend/Ecommerce.Core/`

In plain terms: Core describes the domain vocabulary and the contracts between layers. Entities represent persistent concepts; DTOs define request/response shapes; interfaces define service capabilities; domain helpers hold logic that can be tested without HTTP.

Why it exists: This prevents API handlers and database implementation details from becoming the only place where the domain is defined.

How it fits into the project: For example, `CartEndpoints` accepts cart DTOs and calls `ICartService`; Infrastructure implements that interface using entities and EF Core.

The code:

```
Entities/  Product, Category, Cart, CartItem, Order, OrderItem, Customer, Address, RefreshToken
Dtos/      Catalog, Auth, Cart, Orders, Account, Admin
Domain/    PricingCalculator, CartMergePlanner, PasswordPolicy, SlugGenerator
```

What is happening: A DTO is not automatically a database entity. Keep API input/output contracts separate from the persisted model, and follow the actual types in each feature.

Byte 9: EF Core and the database boundary

Builds on: Byte 8

Connected to: `backend/Ecommerce.Infrastructure/Persistence/EcommerceDbContext.cs`, `Persistence/Configurations/`, `Persistence/Migrations/`

File / Location: `backend/Ecommerce.Infrastructure/Persistence/EcommerceDbContext.cs`

In plain terms: `EcommerceDbContext` is EF Core's gateway to the modern database. Its `DbSet` properties expose queryable collections, while configuration classes define relationships and constraints.

Why it exists: It centralizes database mapping and lets services query/update entities using EF Core.

How it fits into the project: Services use this context; migrations define schema changes; seeding creates development data.

The code:

```
public DbSet<Product> Products => Set<Product>();
public DbSet<Cart> Carts => Set<Cart>();
public DbSet<Order> Orders => Set<Order>();

protected override void OnModelCreating(ModelBuilder modelBuilder)
    => modelBuilder.ApplyConfigurationsFromAssembly(typeof(EcommerceDbContext).Assembly);
```

What is happening: The modern app is code-first EF Core, not the legacy EF6 EDMX Database First setup. Do not edit legacy EDMX files to change the modern database schema.

Byte 10: Database configuration and separation

Builds on: Byte 9

Connected to: `backend/.env.example`, `backend/Ecommerce.Api/appsettings.json`, `backend/docker-compose.dev.yml`, `docs/database.md`

File / Location: Modern backend configuration

In plain terms: The modern app expects a configured `ConnectionStrings:DefaultConnection`. The repository's example configuration names a separate `ShopHubModernDb` database so that legacy data is not accidentally overwritten.

Why it exists: Configuration allows local Docker SQL Server, SQL Server Express, or another supported connection to be selected without hard-coding credentials in source.

How it fits into the project: `ServiceCollectionExtensions` reads the connection string and passes it to EF Core.

The code:

```
var connectionString = configuration.GetConnectionString("DefaultConnection")
    ?? throw new InvalidOperationException("ConnectionStrings:DefaultConnection is not configured.");
```

What is happening: The ZIP includes environment files beyond the example. Treat `.env` and backup files as sensitive; never paste their contents into reports or commit them. Use `.env.example` as a template and rotate any credentials that may have been exposed.

Byte 11: Migrations and seeding

Builds on: Bytes 9–10

Connected to: `backend/Ecommerce.Infrastructure/Persistence/Migrations/`, `Seeding/DatabaseSeeder.cs`, `Seeding/SeedData.cs`

File / Location: `backend/Ecommerce.Infrastructure/`

In plain terms: Migrations evolve the modern database schema, while seeding inserts initial development records such as categories, products, and demo accounts.

Why it exists: A new environment needs a repeatable schema and predictable data before the storefront can be exercised.

How it fits into the project: `InitialiseDatabaseAsync` creates a scope, resolves `DatabaseSeeder`, applies migrations, and seeds.

The code:

```
await seeder.MigrateAsync(ct);
await seeder.SeedAsync(ct);
```

What is happening: Startup-time migration behavior is explicitly registered in Infrastructure. Verify the configured database before running the app: migrations against the wrong connection can change the wrong database.

Byte 12: API endpoint groups

Builds on: Bytes 5–8

Connected to: `backend/Ecommerce.Api/Endpoints/`

File / Location: `backend/Ecommerce.Api/Endpoints/`

In plain terms: Minimal API endpoint classes define the HTTP routes and translate requests into service calls. Feature areas are grouped into files such as `CatalogEndpoints`, `CartEndpoints`, `OrderEndpoints`, `AuthEndpoints`, `AccountEndpoints`, and `AdminEndpoints`.

Why it exists: Route definitions remain organized by business capability rather than accumulating in one giant file.

How it fits into the project: An endpoint receives bound parameters/DTOs, may apply validation/authorization metadata, calls a service, and returns a typed result or HTTP status.

The code:

```
var group = app.MapGroup("/api/cart").WithTags("Cart").AllowAnonymous();
group.MapPost("/items", AddItemAsync);
```

What is happening: Route metadata is part of the API contract. When a UI request fails, check method/path, authentication requirements, request DTO, validation, and service exceptions in that order.

Byte 13: Request validation and error handling

Builds on: Byte 12

Connected to: `backend/Ecommerce.Api/Validation/Validators.cs`, `ValidationFilter.cs`, `Infrastructure/GlobalExceptionHandler.cs`

File / Location: `backend/Ecommerce.Api/Validation/` and `Infrastructure/`

In plain terms: Validation rejects malformed or unacceptable input before it becomes a business operation. The global exception handler maps known failures to HTTP responses rather than letting every endpoint invent its own error format.

Why it exists: Consistent errors make the API safer to use and easier for the frontend to present.

How it fits into the project: Endpoint metadata can attach validated request handling; services still enforce business invariants such as stock and ownership.

The code:

```
HTTP input → request validation → endpoint → service rules → response/error mapping
```

What is happening: Validation is not a replacement for authorization or database constraints. A valid request can still fail because a product is inactive, stock is insufficient, or a resource belongs to another customer.

Byte 14: Authentication design

Builds on: Bytes 6, 12–13

Connected to: `backend/Ecommerce.Api/Auth/`, `backend/Ecommerce.Api/Endpoints/AuthEndpoints.cs`, `backend/Ecommerce.Infrastructure/Services/AuthService.cs`, `JwtTokenService.cs`, `IdentityPasswordHasher.cs`

File / Location: Modern authentication flow

In plain terms: The modern app uses short-lived JWT access tokens and a refresh token stored in an HttpOnly cookie. Password hashing, token creation, login/register rules, and HTTP endpoints are split across different files.

Why it exists: The access token authorizes API requests; the refresh cookie allows the app to obtain a new access token without keeping a long-lived token readable by JavaScript.

How it fits into the project: Auth endpoints call `IAuthService`; token service creates/validates tokens; API authentication middleware validates access tokens; current-user code exposes request identity.

The code:

```
Login/register → AuthService → access token + HttpOnly refresh cookie
API request → Bearer access token → authentication → authorization
```

What is happening: The legacy app instead uses ASP.NET Identity 2/OWIN cookie auth. Do not expect legacy `[Authorize]`/OWIN behavior to be implemented the same way in the modern API.

Byte 15: Refresh-token rotation and frontend recovery

Builds on: Byte 14

Connected to: `backend/Ecommerce.Infrastructure/Services/JwtTokenService.cs`, `backend/Ecommerce.Api/Program.cs`, `frontend/src/api/client.ts`, `frontend/src/stores/auth.ts`

File / Location: Backend token service and `frontend/src/api/client.ts`

In plain terms: The frontend keeps the access token in memory and relies on the HttpOnly cookie for refresh. When an API request gets a 401, Axios can attempt one refresh and retry the original request.

Why it exists: A page reload clears in-memory access-token state, while the cookie can restore a signed-in session. Sharing one in-flight refresh promise avoids simultaneous requests racing to rotate the same refresh token.

How it fits into the project: The auth store restores user state and handles failed refreshes; the API independently enforces authorization.

The code:

```
const client = axios.create({
  baseURL: import.meta.env.VITE_API_BASE_URL ?? '/api',
  withCredentials: true,
})
```

What is happening: Frontend route guards are only a user-experience feature. They must never be treated as the security boundary; server authorization is authoritative.

Byte 16: Roles and admin protection

Builds on: Bytes 12–15

Connected to: `backend/Ecommerce.Api/Auth/`, `backend/Ecommerce.Api/Endpoints/AdminEndpoints.cs`, `frontend/src/router/index.ts`, `frontend/src/stores/auth.ts`

File / Location: API authorization and frontend router

In plain terms: The UI recognizes an Admin role and hides/restricts admin routes accordingly, while the API has its own authorization rules. Registration's documented contract assigns the Customer role rather than accepting a role from the client.

Why it exists: Users must not be able to grant themselves elevated privileges by editing a request or browser state.

How it fits into the project: The router uses `meta.requiresAuth` and `meta.requiresAdmin`; admin API routes enforce server-side policy.

The code:

```
meta: { requiresAuth: true, requiresAdmin: true, title: 'Admin' }
```

What is happening: When debugging an admin issue, inspect both the role returned by `/api/auth/me` and the authorization metadata/policy on the endpoint.

Byte 17: Catalog browsing and product data

Builds on: Bytes 8–12

Connected to: `backend/Ecommerce.Api/Endpoints/CatalogEndpoints.cs`, `backend/Ecommerce.Infrastructure/Services/CatalogService.cs`, `frontend/src/api/catalog.ts`, `frontend/src/stores/catalog.ts`

File / Location: Catalog feature

In plain terms: Catalog services supply category and product information; the frontend API module calls the endpoints and the Pinia catalog store shares the resulting data between views and components.

Why it exists: Catalog behavior such as filtering, sorting, pagination, and home-page selections should be implemented consistently rather than separately in every view.

How it fits into the project: Views such as `HomeView.vue`, `ProductListView.vue`, and `ProductDetailsView.vue` consume catalog state and render components such as `ProductCard.vue` and `ProductList.vue`.

The code:

```
View → catalog store → frontend/src/api/catalog.ts → /api/products or /api/categories
```

What is happening: The API documentation specifies pagination and category hierarchy behavior. Use the endpoint DTOs and `docs/api.md` as the contract instead of guessing response fields.

Byte 18: The cart is persistent, not a browser-only array

Builds on: Bytes 9, 12, 17

Connected to: `backend/Ecommerce.Api/Endpoints/CartEndpoints.cs`, `backend/Ecommerce.Infrastructure/Services/CartService.cs`, `frontend/src/stores/cart.ts`, `frontend/src/api/cart.ts`

File / Location: Cart feature

In plain terms: The modern cart is stored in SQL as `Cart` and `CartItem` records. Guests are identified by an opaque cart cookie; authenticated shoppers are identified by customer ID.

Why it exists: A server-owned cart persists across requests and prevents the browser from being the authority for price or stock.

How it fits into the project: Cart endpoints resolve identity, the service applies changes and recalculates totals, and the Pinia store holds the latest API response for display.

The code:

```
Cart UI → cart store → cart API → CartEndpoints → CartService → EcommerceDbContext
```

What is happening: The service documentation says the authenticated customer ID takes precedence over a stale guest cookie. This prevents a stale anonymous identity from controlling a signed-in customer's cart.

Byte 19: Add-to-cart and authoritative pricing

Builds on: Byte 18

Connected to: `backend/Ecommerce.Infrastructure/Services/CartService.cs`, `backend/Ecommerce.Core/Domain/PricingCalculator.cs`, `frontend/src/components/catalog/ProductCard.vue`

File / Location: `backend/Ecommerce.Infrastructure/Services/CartService.cs`

In plain terms: The client sends product/variant identifiers and quantity; the server looks up the active product, derives the current unit price, updates the cart, checks stock, and saves.

Why it exists: A malicious or stale browser must not be able to set the price it pays.

How it fits into the project: The cart service combines catalog data, pricing rules, cart persistence, and stock checks before returning a new cart DTO.

The code:

```
var unitPrice = PricingCalculator.CurrentUnitPrice(product, variant);
await ValidateStockAsync(cart, ct);
await db.SaveChangesAsync(ct);
```

What is happening: `Product.Price` is documented as the authoritative selling price; `DiscountPercent` is for presentation and must not be applied again to the already-discounted price. The legacy implementation had a documented double-discount defect.

Byte 20: Guest cart merge on login

Builds on: Bytes 14, 18–19

Connected to: `backend/Ecommerce.Core/Domain/CartMergePlanner.cs`, `backend/Ecommerce.Infrastructure/Services/CartService.cs`, `backend/Ecommerce.Api/Endpoints/CartEndpoints.cs`, `frontend/src/stores/auth.ts`

File / Location: `frontend/src/stores/auth.ts` and cart merge endpoint

In plain terms: After login or registration, the frontend requests a merge so guest items are folded into the customer's persistent cart. Duplicate lines, unavailable products, and quantity limits need defined behavior.

Why it exists: Shoppers expect products added before login to remain in their cart afterward.

How it fits into the project: The auth store invokes cart merge; the API requires authentication for the merge endpoint; the service/domain planner applies merge rules and can return warnings.

The code:

```
Guest cart + authenticated cart → merge planner/service → customer's cart + possible warnings
```

What is happening: The auth store's comments call out that the merge must be requested even if the local guest ID is absent, because the API's HttpOnly cookie is the primary identity source. A silent failure can otherwise make the cart appear to disappear after login.

Byte 21: Checkout and order placement

Builds on: Bytes 18–20

Connected to: `backend/Ecommerce.Api/Endpoints/OrderEndpoints.cs`, `backend/Ecommerce.Infrastructure/Services/OrderService.cs`, `backend/Ecommerce.Core/Domain/PricingCalculator.cs`

File / Location: `backend/Ecommerce.Infrastructure/Services/OrderService.cs`

In plain terms: Order placement validates shipping, loads the customer's cart, rechecks product availability, quantity, stock, and current prices, then creates the order and order items.

Why it exists: Checkout is a financial boundary: totals and inventory must be computed from trusted server/database state, not client values.

How it fits into the project: The endpoint handles HTTP and request metadata; `OrderService` implements the use case; EF Core persists the resulting order and stock changes.

The code:

```
var shippingCost = PricingCalculator.ShippingCost(request.ShippingMethod);
var subtotal = PricingCalculator.Subtotal(lines.Select(l => (l.UnitPrice, l.Line.Quantity)));
var total = PricingCalculator.Total(subtotal, shippingCost);
```

What is happening: The service comments document a transaction covering order header, items, stock decrement, and cart finalization. It also uses a database-backed idempotency key so replaying a checkout request can return the existing order instead of charging/creating a second order.

Byte 22: Stock concurrency and idempotency

Builds on: Byte 21

Connected to: `backend/Ecommerce.Infrastructure/Services/OrderService.cs`, `backend/Ecommerce.Infrastructure/Persistence/Configurations/`, `backend/Ecommerce.Infrastructure/Persistence/Migrations/`

File / Location: Order service and entity configuration

In plain terms: Two shoppers may attempt to buy the last unit at the same time, or a client may retry a checkout after a timeout. Concurrency controls and idempotency are designed to protect these cases.

Why it exists: Without them, stock can be oversold or a repeated network submission can create duplicate orders.

How it fits into the project: The order service checks stock, performs updates in a transaction, and uses a concurrency token; the schema/migrations enforce supporting constraints.

The code:

```
Validate cart → transaction → decrement stock → create order/items → finalize cart → commit
```

What is happening: These protections should be tested under failure/retry scenarios, not only happy-path checkout. The service specifically notes SQL Server's retry execution strategy and wraps the transaction in that strategy.

Byte 23: Account profile, addresses, and order history

Builds on: Bytes 14, 21

Connected to: `backend/Ecommerce.Api/Endpoints/AccountEndpoints.cs`, `OrderEndpoints.cs`, `backend/Ecommerce.Infrastructure/Services/AccountService.cs`, `OrderService.cs`, `frontend/src/api/account.ts`, `orders.ts`

File / Location: Account and order feature

In plain terms: Authenticated account pages retrieve and update customer information, addresses, and order history through API contracts.

Why it exists: Account operations need to be separated from anonymous catalog access and scoped to the signed-in customer.

How it fits into the project: Frontend account/order stores and views call API modules; endpoint handlers call account/order services; services use EF Core.

The code:

```
Account/Order view → API module → authenticated endpoint → service → database
```

What is happening: The API docs state that attempts to access another customer's order/address are returned as 404 to avoid confirming that the resource exists. Ownership checks must remain server-side.

Byte 24: Admin catalog and order operations

Builds on: Bytes 16–17, 21

Connected to: `backend/Ecommerce.Api/Endpoints/AdminEndpoints.cs`, `backend/Ecommerce.Infrastructure/Services/AdminService.cs`, `frontend/src/views/admin/`, `frontend/src/api/admin.ts`

File / Location: Admin feature

In plain terms: The admin area exposes management operations separately from the public catalog. The Vue admin views handle product, category, and order screens; the API and AdminService enforce the actual operations.

Why it exists: Administrative actions need elevated permissions and different business rules from ordinary shopping.

How it fits into the project: `AdminProductsView.vue`, `AdminCategoriesView.vue`, and `AdminOrdersView.vue` call the admin API module and display returned data.

The code:

```
Admin view → admin API module → AdminEndpoints → AdminService → database
```

What is happening: Do not rely on the frontend hiding an admin link. Verify authorization policy on every administrative endpoint.

Byte 25: Frontend bootstrap and app shell

Builds on: Bytes 2, 5

Connected to: `frontend/src/main.ts`, `frontend/src/App.vue`, `frontend/src/assets/styles/main.css`, `frontend/src/components/layout/`

File / Location: `frontend/src/main.ts`

In plain terms: `main.ts` creates the Vue app, installs Pinia and Vue Router, loads Bootstrap and project styles, registers the auth-failure callback, and mounts the app.

Why it exists: It initializes the shared frontend infrastructure once before views render.

How it fits into the project: `App.vue` is the root shell; layout components provide shared header/footer; the router chooses the view for each URL.

The code:

```
const app = createApp(App)
app.use(createPinia())
app.use(router)
registerAuthFailureHandler()
app.mount('#app')
```

What is happening: If the app fails before a page appears, check bootstrap imports, router setup, runtime errors, and Vite configuration before debugging an individual component.

Byte 26: Routing and route guards

Builds on: Bytes 14–16, 25

Connected to: `frontend/src/router/index.ts`, `frontend/src/stores/auth.ts`

File / Location: `frontend/src/router/index.ts`

In plain terms: Vue Router maps URLs to lazy-loaded view components and applies metadata-based guards for authenticated, guest-only, and admin pages.

Why it exists: It makes navigation consistent and prevents confusing user journeys, such as sending a signed-in user back to login.

How it fits into the project: On a hard reload, the guard can restore auth state before deciding whether to allow navigation.

The code:

```
if (to.meta.requiresAuth && !auth.isAuthenticated) {
  return { name: 'login', query: { redirect: to.fullPath } }
}
```

What is happening: Guards improve navigation but are not access control. A direct API request must still be rejected by the backend if unauthorized.

Byte 27: Pinia stores and API modules

Builds on: Bytes 17–18, 25

Connected to: `frontend/src/stores/`, `frontend/src/api/`, `frontend/src/types/index.ts`

File / Location: `frontend/src/stores/`

In plain terms: API modules know how to call HTTP endpoints; Pinia stores hold shared UI/application state and expose actions for views to use. TypeScript types describe the expected data shapes.

Why it exists: This keeps network calls and shared state out of individual templates and avoids each view implementing its own version of the same behavior.

How it fits into the project: For example, `cart.ts` calls functions from `api/cart.ts` and stores the returned server cart; `catalog.ts` handles catalog state; `auth.ts` handles identity state.

The code:

```
Vue view/component → Pinia action → API module → Axios client → backend endpoint
```

What is happening: The cart store explicitly treats the API response as source of truth. Client totals are for display only; backend prices and stock remain authoritative.

Byte 28: Horizontal rails and continuous category motion

Builds on: Byte 25

Connected to: `frontend/src/composables/useHorizontalRail.ts`, `frontend/src/test/composables/useHorizontalRail.spec.ts`, `useHorizontalRail.continuous.spec.ts`, `frontend/src/assets/styles/main.css`

File / Location: `frontend/src/composables/useHorizontalRail.ts`

In plain terms: The shared composable supports two rail modes: continuous CSS-animation movement for category tiles and manual scrolling for the featured-products rail.

Why it exists: Continuous motion should look like a steady flow rather than a sudden one-card jump, while other rails can remain user-controlled.

How it fits into the project: The component measures the width of one duplicated track pass and publishes the loop distance to CSS. Manual arrows seek the animation timeline in continuous mode rather than fighting the CSS transform.

The code:

```
continuous?: HorizontalRailContinuous
// Continuous motion is driven by CSS keyframes, not a timer that jumps scrollLeft.
```

What is happening: If motion jumps, stops, or overlaps, inspect duplicated-track markup, measured loop distance, responsive card widths, reduced-motion/accessibility behavior, and the composable tests together.

Byte 29: Error handling, logs, CORS, and rate limits

Builds on: Bytes 6, 12–15

Connected to: `backend/Ecommerce.Api/Program.cs`, `GlobalExceptionHandler.cs`, `ConcurrencyGate.cs`, `backend/.env.example`

File / Location: API middleware and configuration

In plain terms: Cross-cutting middleware handles failures, request logging, allowed frontend origins, throttling, and concurrency-related protections before feature code completes.

Why it exists: These concerns affect many endpoints and should not be duplicated in each handler.

How it fits into the project: `Program.cs` wires the pipeline and configuration; endpoint/service code throws domain-specific errors that the API maps into responses.

The code:

```
Request → exception/logging → CORS/rate limit/concurrency → auth → endpoint → service
```

What is happening: A browser CORS error may happen before the endpoint is reached. A 429 suggests rate limiting; a 401 suggests authentication; a 403 suggests authorization; a 404 can also intentionally conceal resource ownership.

Byte 30: Testing strategy

Builds on: Bytes 8–29

Connected to: `backend/Ecommerce.Tests/`, `frontend/src/test/`, `frontend/e2e/`, `frontend/playwright.config.ts`

File / Location: Test directories

In plain terms: The repository contains backend unit/integration tests, frontend unit/component/store tests, and Playwright end-to-end tests. Each level catches a different class of defect.

Why it exists: Domain calculations can be tested cheaply; API/database interactions need integration tests; complete shopper journeys need browser-level checks.

How it fits into the project: Domain tests exercise helpers such as pricing and cart merging; backend integration tests cover auth, cart/checkout, orders, admin authorization, and rate limiting; frontend tests cover stores/components; E2E tests cover journeys and API docs.

The code:

```
Backend: dotnet test backend/Ecommerce.sln
Frontend: cd frontend && npm test
E2E:      cd frontend && npm run test:e2e
```

What is happening: Treat these as intended commands and verify prerequisites/configuration before running them. Do not claim the suite passes unless it has actually been run in the target environment.

Byte 31: The legacy database and seed files

Builds on: Bytes 3–4

Connected to: `db/schema.sql`, `db/seed_generic.sql`, `db/seed.sql`, `Ecommerce.Data/LegacyEcommerce.edmx`, `Ecommerce.Data/EcommerceDbContext.cs`

File / Location: `db/`, `Ecommerce.Data/`

In plain terms: The legacy app uses SQL Server with an EF6 Database First EDMX model. SQL schema/seed scripts describe the legacy database, and the Data project contains the generated model mapping and repositories.

Why it exists: It provides the original persistence model and reference data for the old MVC application.

How it fits into the project: Legacy services call repositories; repositories use the EF6 context generated from the EDMX. The modern API instead uses EF Core migrations and a separate database.

The code:

```
Legacy Services → legacy repositories → EF6 DbContext / EDMX → LegacyEcommerceDb
```

What is happening: `docs/migration-notes.md` says `seed_generic.sql` is the canonical generic seed and `seed.sql` is superseded history. Confirm the intended source before reseeding anything.

Byte 32: Important migration decisions

Builds on: Bytes 4, 9, 18–22, 31

Connected to: `docs/migration-notes.md`, `docs/IMPLEMENTATION_PLAN.md`, `docs/legacy-implementation-plan.md`

File / Location: `docs/migration-notes.md`

In plain terms: The modern implementation is not a line-for-line translation. It preserves domain concepts but deliberately changes persistence and fixes documented legacy defects.

Why it exists: Legacy behavior included limitations such as a session-only cart, weak password rules, missing admin functionality, and pricing/stock correctness issues.

How it fits into the project: The migration notes are the explanation layer connecting the old and new architectures.

The code:

```
Legacy session cart → modern Cart/CartItem tables
OWIN cookie auth → JWT access token + rotating refresh token
EF6 EDMX → EF Core code-first migrations
```

What is happening: These are architectural decisions, not evidence that both apps share a runtime. When changing behavior, determine whether the requirement applies to the legacy app, the modern port, or both.

Byte 33: End-to-end flow — browse and add to cart

Builds on: Bytes 12, 17–19, 25–27

Connected to: `ProductListView.vue`, `ProductCard.vue`, `stores/catalog.ts`, `stores/cart.ts`, `api/catalog.ts`, `api/cart.ts`, `CatalogEndpoints.cs`, `CartEndpoints.cs`, `CatalogService.cs`, `CartService.cs`

File / Location: Catalog/cart feature

In plain terms: The browser loads product data, renders a product card, and sends product identifiers and quantity when the shopper adds an item. The backend resolves the actual product and price and returns the updated cart.

Why it exists: The UI needs to be responsive while the server remains the source of truth for inventory and pricing.

How it fits into the project:

```
Product view → catalog store/API → CatalogEndpoints → CatalogService → DB
Add click → cart store/API → CartEndpoints → CartService → DB → updated CartDto
```

The code: See `CartService.AddItemAsync` for product lookup, current price, stock validation, and persistence.

What is happening: If the button appears to do nothing, inspect the browser console/network request, API base URL, response status, endpoint validation, and backend logs. Do not debug only the button template.

Byte 34: End-to-end flow — login and restore session

Builds on: Bytes 14–16, 20, 25–27

Connected to: `LoginView.vue`, `LoginForm.vue`, `stores/auth.ts`, `api/auth.ts`, `api/client.ts`, `AuthEndpoints.cs`, `AuthService.cs`, `JwtTokenService.cs`

File / Location: Authentication feature

In plain terms: The login form submits credentials to the API. On success the UI stores the user and in-memory access token, then attempts to merge a guest cart. On a later reload, the frontend can use the refresh cookie to restore access.

Why it exists: The session should survive normal page reloads without persisting the access token in localStorage.

How it fits into the project:

```
Login form → auth store/API → AuthEndpoints → AuthService → token service
           ← user/access token + HttpOnly refresh cookie
```

The code: See `frontend/src/api/client.ts` for refresh/retry and `frontend/src/stores/auth.ts` for restore/merge behavior.

What is happening: Distinguish credential failure (401), refresh failure, CORS/cookie configuration, and route-guard behavior. They can look similar in the UI but have different causes.

Byte 35: End-to-end flow — checkout

Builds on: Bytes 18–23, 29

Connected to: `CheckoutView.vue`, `CheckoutForm.vue`, `api/orders.ts`, `OrderEndpoints.cs`, `OrderService.cs`, `PricingCalculator.cs`, EF Core configurations

File / Location: Order feature

In plain terms: Checkout submits shipping/billing details and a shipping method; the backend validates the cart against current products and stock, computes totals, then commits order creation and inventory changes together.

Why it exists: An order must be internally consistent even if the browser retries or a database operation fails partway through.

How it fits into the project:

```
Checkout form → order API → OrderEndpoints → OrderService
→ validate → transaction → order/items + stock decrement + cart finalization → response
```

The code: `backend/Ecommerce.Infrastructure/Services/OrderService.cs`

What is happening: When debugging, check the API response and server exception, then inspect stock, shipping method, cart ownership, transaction behavior, and idempotency key. A successful UI click alone does not prove an order was persisted.

Byte 36: Security and configuration gotchas

Builds on: Bytes 10, 14–16, 29

Connected to: `.gitignore`, `backend/.env.example`, `backend/.env`, `backend/.env.backup`, appsettings files

File / Location: Configuration files

In plain terms: Secrets and runtime configuration are not ordinary source code. The repository contains `.env`-named files as well as a safe example file; the example explicitly says it should not contain real secrets.

Why it exists: Credentials, JWT signing keys, database passwords, and seeded account passwords must vary by environment and remain private.

How it fits into the project: Environment variables and configuration providers feed the connection string, JWT validation, CORS, and rate limiting.

The code:

```
Safe template: .env.example
Private runtime values: .env / user-secrets / deployment environment
```

What is happening: Do not copy real secret values into documentation, source control, or prompts sent to third-party agents. If a secret was committed or included in a shared ZIP, treat it as exposed and rotate it.

Final project map

Architecture

```
LEGACY APPLICATION
Browser → ASP.NET MVC 5 / Razor (Ecommerce.Web)
        → Ecommerce.Services
        → Ecommerce.Data (EF6 Database First / repositories)
        → LegacyEcommerceDb (SQL Server)

MODERN SHOPHUB APPLICATION
Browser → Vue 3 SPA (views/components, Pinia, Vue Router, Axios)
        → ASP.NET Core Minimal API (.NET 10)
        → Ecommerce.Core (entities, DTOs, service contracts, domain helpers)
        → Ecommerce.Infrastructure (services, EF Core DbContext, migrations, seeding)
        → ShopHubModernDb (separate SQL Server database)
```

Major components

Component	Responsibility	Important files
Modern API startup	Middleware, DI, auth, CORS, rate limits, endpoint mapping	`backend/Ecommerce.Api/Program.cs`
Domain/contracts	Entities, DTOs, interfaces, pure helpers	`backend/Ecommerce.Core/`
Persistence	EF Core context, entity configuration, migrations	`backend/Ecommerce.Infrastructure/Persistence/`
Business services	Auth, catalog, cart, order, account, admin	`backend/Ecommerce.Infrastructure/Services/`
HTTP endpoints	Route definitions and response mapping	`backend/Ecommerce.Api/Endpoints/`
Frontend bootstrap/routing	Initialize Vue and map URLs to views	`frontend/src/main.ts`, `router/index.ts`
Frontend state/API	Shared state and HTTP calls	`frontend/src/stores/`, `frontend/src/api/`
UI components	Catalog, cart, checkout, account, admin	`frontend/src/components/`, `frontend/src/views/`
Legacy UI/business/data	MVC/Razor, services, EF6 repositories	`Ecommerce.Web/`, `Ecommerce.Services/`, `Ecommerce.Data/`, `Ecommerce.Core/`
Migration/reference docs	Explains architectural decisions and legacy behavior	`docs/`
Tests	Unit, integration, component, E2E	`backend/Ecommerce.Tests/`, `frontend/src/test/`, `frontend/e2e/`

Important data flows

1. **Catalog:** Vue view → catalog store/API → catalog endpoint → catalog service → database.

2. **Cart mutation:** cart UI → Pinia cart store → Axios API module → cart endpoint → `CartService` → database.

3. **Login/refresh:** login UI → auth endpoint/service → access token + HttpOnly refresh cookie; Axios refreshes access token when appropriate.

4. **Guest cart merge:** successful login → auth store → authenticated merge endpoint → cart service/planner → customer cart.

5. **Checkout:** checkout view → order endpoint → `OrderService` → transaction → order/items, stock updates, cart finalization.

6. **Admin operation:** admin view → admin API module → protected endpoint → `AdminService` → database.

7. **Legacy request:** MVC controller → legacy service → EF6 repository/context → legacy SQL Server database.

High-value files to read next

File	Why it matters
`README.md`	Setup and high-level repository instructions
`docs/architecture.md`	Modern architecture and middleware
`docs/legacy-architecture.md`	Original four-layer architecture
`docs/migration-notes.md`	Intentional differences and corrected legacy defects
`backend/Ecommerce.Api/Program.cs`	Startup and request pipeline
`backend/Ecommerce.Infrastructure/ServiceCollectionExtensions.cs`	Service/database registrations

File	Why it matters
`backend/Ecommerce.Infrastructure/Services/CartService.cs`	Persistent cart behavior
`backend/Ecommerce.Infrastructure/Services/OrderService.cs`	Checkout correctness and transaction
`frontend/src/api/client.ts`	Shared HTTP client and refresh retry
`frontend/src/stores/auth.ts`	Authentication state and guest-cart merge
`frontend/src/stores/cart.ts`	Frontend cart state
`frontend/src/composables/useHorizontalRail.ts`	Continuous category rail behavior
`backend/Ecommerce.Tests/` and `frontend/e2e/`	How important behavior is verified

Key concepts to be able to explain after studying

• Why the repository has both a legacy MVC app and a modern SPA/API app.

• The difference between EF6 Database First and EF Core code-first migrations.

• How dependency injection connects service interfaces to implementations.

• How the API contract connects frontend views/stores to backend endpoint/service code.

• Why prices, totals, stock, ownership, and authorization must be enforced server-side.

• How guest and authenticated carts are identified and merged.

• How refresh-token rotation and frontend retry logic work together.

• Why checkout needs transactions, concurrency protection, and idempotency.

• Why the modern database must remain separate from the legacy database.

• How unit, integration, frontend, and end-to-end tests complement one another.

Scope and limitations

This guide is based on the repository documentation and representative source files inspected from the ZIP. It is a guided learning sequence for the major system responsibilities, not a byte for every source file. Generated files, compiled binaries, dependency folders, and static image assets are intentionally not explained individually. Any runtime status, test pass count, or deployment success must be verified by running the commands in the intended environment.